

Desafio 1 – LÓGICA DE PROGRAMAÇÃO – TEMA 1

ArenaCode Championship – Sistema de Gerenciamento de Campeonato de E-Sports

1. Contextualização

O desenvolvimento de software não consiste apenas em conhecer comandos de uma linguagem de programação. Um programador precisa ser capaz de analisar uma situação, identificar informações relevantes, estabelecer regras, decompor um problema complexo em partes menores e construir uma solução computacional organizada.

Durante a disciplina de “Algoritmo e Programação”, diversos conceitos foram estudados individualmente, como variáveis, estruturas condicionais, estruturas de repetição, vetores, matrizes, registros, procedimentos, funções, passagem de parâmetros e recursividade.

Neste desafio, esses conhecimentos deverão ser utilizados de maneira **integrada**.

Você não receberá uma sequência de pequenas tarefas independentes. Receberá um problema maior, semelhante ao que ocorre no desenvolvimento de um sistema real, e deverá decidir como organizar os dados, quais estruturas utilizar e como dividir o algoritmo em diferentes módulos.

2. Texto-base

Uma instituição universitária realizará um campeonato interno de jogos eletrônicos denominado **ArenaCode Championship**. Diversas equipes participarão da competição. Cada equipe possui informações próprias, participa de partidas contra outras equipes e acumula pontos de acordo com seus resultados.

A organização inicialmente realizava todo o controle utilizando anotações manuais. Entretanto, com o aumento da quantidade de equipes e partidas, começaram a ocorrer problemas relacionados ao cálculo da classificação, identificação das equipes, armazenamento dos resultados e determinação das posições no campeonato. A comissão organizadora decidiu solicitar o desenvolvimento de um algoritmo capaz de controlar as principais informações da competição.

Como o objetivo do projeto é avaliar conhecimentos de **Lógica de Programação**, o sistema deverá ser desenvolvido integralmente utilizando **VisuAlg**, sem banco de dados ou interface gráfica.

3. Situação-problema

Você foi responsável pelo desenvolvimento do algoritmo que realizará o gerenciamento do campeonato.

O sistema deverá permitir cadastrar equipes, registrar resultados das partidas, calcular automaticamente a pontuação, consultar informações e apresentar a classificação geral da competição.

Cada equipe deverá possuir, no mínimo, as seguintes informações:

- código;
- nome da equipe;
- nome do capitão;
- quantidade de vitórias;
- quantidade de empates;
- quantidade de derrotas;
- pontos marcados;
- pontos sofridos;
- saldo de pontos;
- pontuação no campeonato.

Considere que:

- vitória vale **3 pontos**;
- empate vale **1 ponto**;
- derrota vale **0 pontos**.

Esses valores deverão ser definidos no algoritmo por meio de **constantes**, evitando números fixos espalhados pelo código. O campeonato deverá permitir o cadastro de uma quantidade limitada de equipes, definida previamente pelo desenvolvedor.

4. Estruturas de dados obrigatórias

O sistema deverá trabalhar com diferentes formas de armazenamento de dados.

4.1 Registro

As informações de cada equipe deverão ser agrupadas utilizando uma estrutura do tipo **registro**.

O registro deverá representar logicamente uma equipe e seus respectivos dados.

4.2 Vetor

As equipes cadastradas deverão ser armazenadas em um **vetor de registros**.

O vetor deverá permitir que o sistema mantenha simultaneamente as informações de todas as equipes participantes.

4.3 Matriz

Uma **matriz** deverá ser utilizada para representar informações relacionadas aos confrontos realizados entre as equipes. A equipe deverá decidir a melhor forma de utilizar essa matriz.

Uma possibilidade é utilizar as linhas e colunas para representar as equipes e armazenar alguma informação referente aos confrontos realizados.

Exemplo conceitual:

Equipe	Alpha	ByteForce	CodeWarriors	DevTeam
Alpha	-	1	0	1
ByteForce	1	-	1	0
CodeWarriors	0	1	-	1
DevTeam	1	0	1	-

A implementação exata deverá ser definida pelos alunos e explicada por meio de comentários no código.

5. Menu principal

Após a inicialização, o sistema deverá apresentar continuamente um menu semelhante ao seguinte:

ARENA CODE CHAMPIONSHIP

1. Cadastrar equipe
2. Listar equipes
3. Registrar resultado de partida
4. Consultar equipe
5. Exibir classificação geral
6. Exibir estatísticas do campeonato
7. Consultar confrontos
8. Executar recurso recursivo
9. Encerrar sistema

A seleção das funcionalidades deverá obrigatoriamente utilizar uma estrutura de **múltipla escolha**. O menu deverá permanecer disponível até que o usuário escolha explicitamente a opção de encerramento.

6. Cadastro de equipes

O sistema deverá permitir o cadastro das equipes participantes. Cada equipe deverá possuir um código único. O algoritmo não poderá permitir:

- códigos duplicados;
- nomes vazios;
- cadastro acima da capacidade máxima definida;
- valores inválidos nos campos solicitados.

No momento do cadastro, as estatísticas da equipe deverão ser inicializadas adequadamente. O cadastro deverá ser implementado utilizando uma **sub-rotina**.

7. Registro das partidas

O usuário deverá informar:

- código da primeira equipe;
- código da segunda equipe;
- pontuação da primeira equipe;
- pontuação da segunda equipe.

O sistema deverá verificar se as duas equipes existem. Também deverá impedir que uma equipe jogue contra ela própria. Depois da validação, o algoritmo deverá determinar automaticamente se ocorreu:

- vitória da primeira equipe;
- vitória da segunda equipe;
- empate.

As estatísticas das duas equipes deverão ser atualizadas automaticamente. Isso inclui:

- vitórias;
- empates;
- derrotas;
- pontos marcados;
- pontos sofridos;
- saldo;
- pontuação no campeonato.

Os operadores **aritméticos, relacionais e lógicos** deverão ser utilizados adequadamente para realizar essas operações e validações.

8. Consulta de equipes

O sistema deverá permitir pesquisar uma equipe pelo seu código. Caso seja encontrada, deverão ser apresentadas suas informações completas. Caso contrário, deverá ser apresentada uma mensagem informando que a equipe não foi localizada. A busca deverá ser implementada utilizando uma **função**, que deverá receber informações por parâmetro e retornar o resultado da pesquisa.

9. Classificação geral

Uma das funcionalidades mais importantes do sistema será a geração da tabela de classificação. Antes da apresentação, o vetor contendo as equipes deverá ser **ordenado**.

A classificação deverá considerar inicialmente:

1. maior quantidade de pontos;
2. maior quantidade de vitórias;
3. maior saldo de pontos;
4. maior quantidade de pontos marcados.

Portanto, caso duas equipes possuam a mesma pontuação, deverão ser utilizados os critérios seguintes para desempate.

Exemplo:

Pos.	Equipe	P	V	E	D	PM	PS	SG
1	CodeWarriors	15	5	0	1	42	20	22
2	Alpha	13	4	1	1	35	21	14
3	ByteForce	10	3	1	2	30	28	2

O algoritmo de ordenação deverá ser implementado pelos próprios alunos.

Não será suficiente simplesmente apresentar as equipes na ordem em que foram cadastradas.

10. Estatísticas do campeonato

O sistema deverá possuir uma opção capaz de apresentar estatísticas gerais.

Deverão ser calculadas, no mínimo:

- quantidade de equipes cadastradas;
- quantidade total de partidas realizadas;
- quantidade de vitórias;
- quantidade de empates;
- média de pontos marcados por partida;
- equipe com maior número de vitórias;
- equipe com maior pontuação;
- equipe com maior saldo;
- equipe com maior quantidade de pontos marcados.

Essas informações deverão ser calculadas utilizando estruturas de repetição, acumuladores, contadores e comparações.

11. Sub-rotinas

O algoritmo deverá obrigatoriamente ser dividido em **procedimentos e funções**. Não será aceita uma solução em que toda a lógica esteja concentrada no algoritmo principal.

Entre outras possibilidades, poderão existir sub-rotinas responsáveis por:

- cadastrar equipe;
- localizar equipe;
- registrar partida;
- calcular saldo;

- calcular pontuação;
- ordenar classificação;
- exibir classificação;
- exibir estatísticas;
- validar dados;
- exibir informações de uma equipe.

Aqui analisar quais operações devem ser implementadas como **procedimentos** e quais são mais adequadas como **funções**.

12. Passagem de parâmetros

As sub-rotinas deverão utilizar adequadamente **passagem de parâmetros**.

Aqui é fundamental compreender quando uma informação precisa apenas ser enviada para uma sub-rotina e quando uma alteração realizada dentro dela precisa refletir na estrutura principal do algoritmo. Sempre que possível, evite utilizar variáveis globais apenas para facilitar o acesso às informações.

13. Variáveis locais e globais

O algoritmo deverá possuir exemplos adequados de:

- variáveis globais;
- variáveis locais.

As variáveis globais deverão representar informações que realmente precisem estar disponíveis para diferentes partes deste algoritmo.

As variáveis utilizadas apenas temporariamente dentro de procedimentos e funções deverão ser declaradas localmente.

O objetivo é demonstrar compreensão sobre **escopo de variáveis**, e não simplesmente utilizar variáveis globais para tudo.

14. Recursividade

O sistema deverá possuir pelo menos **uma função ou procedimento recursivo**. A utilização da recursividade deverá possuir:

- caso-base;
- chamada recursiva;
- condição que garanta o encerramento.

Como parte do desafio, os alunos deverão implementar recursivamente uma funcionalidade relacionada ao campeonato. Uma possibilidade é apresentar uma **contagem regressiva para o início da competição**.

Exemplo:

Iniciando campeonato em...
5, 4, 3, 2, 1 ...
CAMPEONATO INICIADO!

Entretanto, a equipe poderá propor outra utilização coerente para a recursividade, desde que seja capaz de explicar sua implementação.

15. Constantes

Deverão ser utilizadas constantes para representar valores que não serão alterados durante a execução.

Exemplos:

```
PONTOS_VITORIA = 3
PONTOS_EMPATE = 1
PONTOS_DERROTA = 0
MAX_EQUIPES = 10
```

Evite repetir valores fixos diretamente em diferentes partes do algoritmo.

16. Requisitos técnicos obrigatórios

A solução deverá apresentar obrigatoriamente:

1. variáveis simples;
2. vetor e matriz;
3. registro e vetor de registros;
4. constantes;
5. operadores aritméticos, relacionais e lógicos;
6. estrutura condicional;
7. estrutura condicional composta e/ou encadeada;
8. estrutura de múltipla escolha e de repetição;
9. estruturas de repetição aninhadas;
10. procedimento;
11. função;
12. passagem de parâmetros;
13. variáveis locais;
14. variáveis globais;
15. algoritmo de ordenação;
16. recursividade;
17. validação de dados de entrada.

A presença desses recursos deverá fazer sentido dentro da solução.

Não será considerada adequada a utilização artificial de determinada estrutura apenas para cumprir um requisito.

17. Restrições

O algoritmo deverá ser desenvolvido integralmente no **VisuAlg**.

Não deverão ser utilizadas:

- linguagens externas;
- banco de dados;
- bibliotecas externas;
- códigos prontos que substituam a lógica solicitada;
- recursos que eliminem a necessidade de implementação dos algoritmos estudados.

O objetivo não é construir um sistema comercial, mas demonstrar domínio sobre os fundamentos de **Lógica de Programação e Programação Estruturada**.

18. Testes obrigatórios

O algoritmo deverá ser testado em diferentes cenários.

Entre os testes, deverão existir situações como:

- tentativa de cadastrar código duplicado;
- tentativa de registrar partida com equipe inexistente;
- tentativa de registrar uma equipe contra ela própria;
- partida com vencedor;
- partida empatada;
- equipes empatadas em pontos na classificação;
- aplicação dos critérios de desempate;
- tentativa de acessar opção inexistente no menu;
- execução da funcionalidade recursiva.

Os testes deverão demonstrar que o algoritmo não funciona apenas no chamado "**caminho feliz**", mas também consegue tratar situações incorretas ou excepcionais.

19. Organização do código

O código deverá apresentar organização e legibilidade.

Utilize:

- nomes significativos para variáveis;
- indentação;
- comentários;
- divisão adequada em sub-rotinas;
- constantes para valores fixos;
- estruturas adequadas para cada problema.

Evite códigos excessivamente repetidos.

Quando determinada operação for executada várias vezes, analise se ela pode ser transformada em uma função ou procedimento.

20. Chamada para ação

Seu desafio

A comissão organizadora do **ArenaCode Championship** precisa abandonar os controles manuais e utilizar uma solução algorítmica capaz de organizar corretamente a competição.

Sua missão é **projetar, desenvolver, testar e apresentar um algoritmo completo em VisuAlg** que resolva o problema apresentado. Antes de começar a programar, analise cuidadosamente a situação.

Pergunte-se:

- Quais informações precisam ser armazenadas?
- Qual estrutura de dados representa melhor cada informação?
- Quais operações se repetem e deveriam ser transformadas em sub-rotinas?
- Quais informações devem ser enviadas por parâmetros?
- Quais variáveis realmente precisam ser globais?
- Como a classificação será ordenada?
- Como impedir estados inválidos no sistema?
- Onde a recursividade pode ser aplicada corretamente?

O desafio não consiste apenas em fazer o algoritmo funcionar. Uma solução que produz resultados corretos, mas possui código desorganizado, duplicado ou estruturas utilizadas de maneira inadequada, não demonstra domínio completo dos conceitos estudados.

O objetivo é desenvolver uma solução **logicamente correta, estruturada, modular, legível e justificável**.

DESAFIO FINAL

Ao concluir o desenvolvimento, você deverá ser capaz de explicar seu próprio algoritmo.

Você poderá ser questionado sobre qualquer trecho do código, incluindo:

- por que escolheu determinado tipo de variável;
- como funciona o vetor de registros;
- como a matriz foi utilizada;
- como funciona o algoritmo de ordenação;
- por que determinada operação foi implementada como função ou procedimento;
- quais parâmetros são utilizados pelas sub-rotinas;
- quais variáveis são locais e quais são globais;
- como os operadores lógicos determinam determinadas decisões;
- qual é o caso-base da recursividade;
- o que aconteceria se determinado trecho do código fosse alterado.

Um algoritmo não está verdadeiramente concluído quando apenas executa sem erros, está concluído quando seu desenvolvedor consegue compreender, testar, justificar e explicar a lógica utilizada para construí-lo.

Entrega:

- Individual.
- Criar uma pasta **DESAFIO1** no repositório da disciplina no GitHub.
- Salvar nesta pasta o código em um único arquivo em **formato .alg**
- Salvar também um arquivo em **formato .txt** contendo o link do vídeo explicativo.
- O vídeo deve ser gravado e submetido ao YouTube, Google Drive ou outro serviço de armazenamento em Cloud e compartilhar o link.